

소스 변경 상세 — 무엇이 어떻게 바뀌었나

①번 문서의 파일 14개가 실제로 어떻게 바뀌는지 전·후로 보여 줍니다. 파일을 통째로 교체하실 거면 6절만 보셔도 되고, 손으로 고치거나 대조 검수를 하실 때 2~5절을 쓰십시오. 신규 자바 6개의 소스 전문은 맨 끝 「별첨」에 있습니다.

작성일 2026-09-08 (확정본) 기준 내부망 반입 zip(2026-09-07) 대비 선행 문서 ① 스키마 변경

검증 `mvn -o clean test` 62건 통과

1 왜 자바까지 바꾸나 — TypeHandler

MyBatis의 `resultMap` 은 자바 필드와 DB 컬럼을 잇는 번역표이고, `TypeHandler`는 그 번역표가 값을 어떻게 바꿀지 정하는 규칙입니다. 표준이 요구하는 값(문자열 시각·2자리 코드)과 자바가 쓰는 값이 다르므로, 변환을 세 클래스에 가두어 도메인·서비스·화면·테스트를 지켰습니다.

TypeHandler	자바가 쓰는 값	DB 에 들어가는 값
<code>LocalDateTimeStringTypeHandler</code>	<code>LocalDateTime</code>	<code>'20260908103000'</code>
<code>ClassificationStatusTypeHandler</code>	<code>"UNCLASSIFIED"</code>	<code>'01'</code>
<code>InstitutionCategoryTypeHandler</code>	<code>"지방자치단체"</code>	<code>'01'</code>

규칙 두 개만 지키면 됩니다

- `resultMap` 과 `#{...}` 양쪽에 적는다. 두 코드 핸들러가 모두 자바 `String` 을 다뤄 자동 등록을 쓸 수 없습니다. 한쪽만 적으면 읽거나 쓰기 중 한 방향만 변환돼 조용히 어긋납니다.
- `SQL` 에 직접 박은 값은 코드로 적는다. `TypeHandler` 는 파라미터와 결과에만 걸립니다. `SET` 분류 상태구분 = `'04'` 자리에 `'DELETED'` 를 적으면 오류 없이 0건이 바뀝니다.

2 신규 자바 6개 — 무엇을 하는 파일인가

파일	역할	고칠 일이 생기면
code/InstitutionCategory.java	기관영업분류코드 코드↔이름 5쌍. 화면 드롭다운 목록의 유일한 출처	분류를 늘리거나 이름을 바꿀 때 이 파일만
code/ClassificationStatus.java	분류상태 코드↔값 4쌍 + 상수	상태를 늘릴 때(도메인도 함께)
config/CodeTypeHandler.java	코드 변환 공통 뼈대(추상). 모르는 값이면 예외를 던진다	거의 없음
config/InstitutionCategoryTypeHandler.java	위 뼈대 + 기관영업분류코드 표	없음
config/ClassificationStatusTypeHandler.java	위 뼈대 + 분류상태 표	없음
config/LocalDateTimeStringTypeHandler.java	yyyyMMddHHmmss 변환. 읽을 때 trim() 한다	일시 형식이 바뀌면

모르는 값이 오면 조용히 넘기지 않습니다

ClassificationStatusTypeHandler: 코드로 옮길 수 없는 값이다 - '지자체'. 아는 값: [...]
 InstitutionCategoryTypeHandler: DB 에 모르는 코드가 들어 있다 - '99'. 아는 코드: [01, 02, ...]
 일시 형식이 YYYYMMDDHH24MISS 가 아니다 - '2026-09-08 10:30:00'

코드가 어긋나면 증상이 "저장은 됐는데 조회가 안 됨"처럼 엉뚱하게 나타나므로, **그 자리에서 예외로 멈추게** 했습니다. 위 메시지가 보이면 원인이 바로 드러납니다.

3 UploadedFileMapper.xml

3-1. resultMap 9행 전부 + TypeHandler 4곳

```
전 <id      property="id"           column="ID"/>
<result property="originalName"    column="ORIGINAL_NAME"/>
<result property="storedPath"      column="STORED_PATH"/>
<result property="year"            column="FILE_YEAR"/>
<result property="institutionName" column="INSTITUTION_NAME"/>
<result property="category"        column="CATEGORY"/>
<result property="status"          column="STATUS"/>
<result property="uploadedAt"      column="UPLOADED_AT"/>
<result property="classifiedAt"    column="CLASSIFIED_AT"/>
```

```
후 <id      property="id"           column="업로드파일일련번호"/>
<result property="institutionName" column="기관명"/>
<result property="category"        column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="year"            column="문서년"/>
<result property="status"          column="분류상태구분"
      typeHandler="com.kb.uploader.config.ClassificationStatusTypeHandler"/>
<result property="originalName"    column="원본파일명"/>
<result property="uploadedAt"      column="업로드일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="classifiedAt"    column="분류일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
<result property="storedPath"      column="저장경로"/>
```

3-2. <sql id="값목록"> 신설 — 파라미터 쪽 표기가 길어 묶었다

```
신설 <sql id="값목록">
    #{institutionName},
    #{category,typeHandler=com.kb.uploader.config.InstitutionCategoryTypeHandler},
    #{year},
    #{status,typeHandler=com.kb.uploader.config.ClassificationStatusTypeHandler},
    #{originalName},
    #{uploadedAt,typeHandler=com.kb.uploader.config.LocalDateTimeStringTypeHandler},
    #{storedPath}
</sql>
```

3-3. insert oracle — 내부망이 실제로 쓰는 쪽

```
전 SELECT UPLOADED_FILE_SEQ.NEXTVAL FROM DUAL
INSERT INTO UPLOADED_FILE
    (ID, ORIGINAL_NAME, STORED_PATH, FILE_YEAR, INSTITUTION_NAME, CATEGORY, STATUS, UPLOADED_AT)
VALUES
    (#{id}, #{originalName}, #{storedPath}, #{year}, #{institutionName}, #{category}, #{status}, #{uploadedAt})
```

```
후 SELECT SQ_KGI_AGENTFILETABLE.NEXTVAL FROM DUAL
INSERT INTO TSKGIAF01
    (업로드파일일련번호, 기관명, 기관영업분류코드, 문서년, 분류상태구분, 원본파일명, 업로드일시, 저장경로)
VALUES
    (#{id}, <include refid="값목록"/>)
```

⚠ 시퀀스 이름도 바뀝니다. 여기가 틀리면 ORA-02289 로 업로드만 실패합니다.

3-4. update

```
전 UPDATE UPLOADED_FILE
SET STORED_PATH = #{storedPath}, INSTITUTION_NAME = #{institutionName},
    FILE_YEAR = #{year}, CATEGORY = #{category},
    STATUS = #{status}, CLASSIFIED_AT = #{classifiedAt}
WHERE ID = #{id}
```

```
후 UPDATE TSKGIAF01
SET 저장경로 = #{storedPath},
    기관명 = #{institutionName},
    문서년 = #{year},
    기관영업분류코드 = #{category,typeHandler=...InstitutionCategoryTypeHandler},
    분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler},
    분류일시 = #{classifiedAt,typeHandler=...LocalDateTimeStringTypeHandler}
WHERE 업로드파일일련번호 = #{id}
```

3-5. softDeleteById · rejectById 값을 코드로

```
전 UPDATE UPLOADED_FILE SET STATUS = 'DELETED' WHERE ID = #{id}
UPDATE UPLOADED_FILE SET STATUS = 'REJECTED' WHERE ID = #{id}
```

```
후 UPDATE TSKGIAF01 SET 분류상태구분 = '04' WHERE 업로드파일일련번호 = #{id}
UPDATE TSKGIAF01 SET 분류상태구분 = '03' WHERE 업로드파일일련번호 = #{id}
```

⚠ **여기가 이번 변경에서 가장 놓치기 쉬운 자리입니다.** 리터럴은 TypeHandler 를 타지 않아 'DELETED' 를 그대로 두면 오류 없이 아무 행동 안 바꿉니다.

3-6. findByStatus 정렬 신설 + 파라미터 변환

```
전 SELECT * FROM UPLOADED_FILE WHERE STATUS = #{status}
```

```
후 SELECT * FROM TSKGIAF01
WHERE 분류상태구분 = #{status,typeHandler=...ClassificationStatusTypeHandler}
ORDER BY 업로드파일일련번호 DESC
```

종전에는 ORDER BY 가 없었습니다. 목록 순서를 DB 에 맡긴 셈이라 삽입순처럼 보였을 뿐 보장이 아니었고, 인덱스가 붙는 순간 순서가 바뀝니다. 이 문장은 앱에서 가장 많이 쓰입니다 — 파일처리상태 화면·분류완료 목록 화면·재분류 잡·InstitutionService 두 곳, 모두 여섯 자리.

3-7. findRecent 정렬 기준을 PK 로

```
전 SELECT * FROM (
    SELECT * FROM UPLOADED_FILE ORDER BY UPLOADED_AT DESC
) WHERE ROWNUM <= #{limit}
```

```
후 SELECT * FROM (
    SELECT * FROM TSKGIAF01 ORDER BY 업로드파일일련번호 DESC
) WHERE ROWNUM <= #{limit}
```

업로드일시가 문자열 초 단위라 같은 초에 여러 건이면 순서가 뒤섞입니다. PK 는 그 경우에도 정확하고, PK 인덱스가 정렬을 그대로 처리합니다.

```
전 SELECT COUNT(*) FROM UPLOADED_FILE
SELECT COUNT(*) FROM UPLOADED_FILE WHERE STATUS = #{status}
... WHERE INSTITUTION_NAME = '알수없음' AND STATUS = 'CLASSIFIED' ORDER BY UPLOADED_AT DESC
... WHERE INSTITUTION_NAME LIKE '%' || #{keyword} || '%' ORDER BY UPLOADED_AT DESC
... AND (#{year} IS NULL OR FILE_YEAR = #{year})
... AND (#{keyword} IS NULL OR ORIGINAL_NAME LIKE '%' || #{keyword} || '%')
```

```
후 SELECT COUNT(*) FROM TSKGIAF01
SELECT COUNT(*) FROM TSKGIAF01 WHERE 분류상태구분 = #{status,typeHandler=...}
... WHERE 기관명 = '알수없음' AND 분류상태구분 = '02' ORDER BY 업로드일시 DESC
... WHERE 기관명 LIKE '%' || #{keyword} || '%' ORDER BY 업로드일시 DESC
... AND (#{year} IS NULL OR 문서년 = #{year})
... AND (#{keyword} IS NULL OR 원본파일명 LIKE '%' || #{keyword} || '%')
```

검색·페이징은 일부러 업로드일시 정렬을 유지했습니다

인덱스 (기관명, 업로드일시) 의 두 번째 열이 업로드일시라, 기관 입력이 드롭다운(정확 일치)이 되는 날 정렬까지 인덱스가 받게 하려는 것입니다. 여기를 일련번호로 바꾸면 그 이점이 사라집니다.

4 InstitutionMapper.xml

4-1. resultMap

```
전 <id      property="id"          column="ID"/>
<result property="name"          column="NAME"/>
<result property="category"      column="CATEGORY"/>
<result property="modifiedAt"    column="MODIFIED_AT"/>
```

```
후 <id      property="id"          column="기관일련번호"/>
<result property="name"          column="기관명"/>
<result property="category"      column="기관영업분류코드"
      typeHandler="com.kb.uploader.config.InstitutionCategoryTypeHandler"/>
<result property="modifiedAt"    column="수정일시"
      typeHandler="com.kb.uploader.config.LocalDateTimeStringTypeHandler"/>
```

4-2. 나머지 7문장

```
전 SELECT INSTITUTION_SEQ.NEXTVAL FROM DUAL
INSERT INTO INSTITUTION (ID, NAME, CATEGORY, MODIFIED_AT) VALUES ({id}, {name}, {category}, {modifiedAt})
INSERT INTO INSTITUTION (NAME, CATEGORY, MODIFIED_AT)                                -- mysql
UPDATE INSTITUTION SET NAME = {name}, CATEGORY = {category}, MODIFIED_AT = {modifiedAt} WHERE ID = {id}
DELETE FROM INSTITUTION WHERE ID = {id}
DELETE FROM INSTITUTION
SELECT * FROM INSTITUTION WHERE NAME = {name}
SELECT * FROM INSTITUTION ORDER BY NAME ASC
```

```
후 SELECT SQ_KGI_AGENTGIGANTABLE.NEXTVAL FROM DUAL
INSERT INTO TSKGIAF02 (기관일련번호, 기관명, 기관영업분류코드, 수정일시) VALUES ({id}, <include refid="값목록"/>)
INSERT INTO TSKGIAF02 (기관명, 기관영업분류코드, 수정일시)                                -- mysql
UPDATE TSKGIAF02 SET 기관명 = {name}, 기관영업분류코드 = {category,typeHandler=...},
                    수정일시 = {modifiedAt,typeHandler=...} WHERE 기관일련번호 = {id}
DELETE FROM TSKGIAF02 WHERE 기관일련번호 = {id}
DELETE FROM TSKGIAF02
SELECT * FROM TSKGIAF02 WHERE 기관명 = {name}
SELECT * FROM TSKGIAF02 ORDER BY 기관명 ASC
```

`findByName` 이 이 앱에서 가장 중요한 문장입니다

분류가 여기서 결정됩니다 — 업로드된 파일의 기관명으로 이 문장을 돌려 못 찾으면 파일이 미분류로 남습니다. 틀리면 오류 없이 “분류가 하나도 안 되는” 증상으로만 나타납니다.

TSKGIAF02 에 컬럼 4개가 늘었지만 매퍼는 바뀌지 않습니다

협약기관상태구분 · 입찰공고상태 · 입찰공고시작 · 입찰공고마감 이 신설됐으나 애플리케이션이 읽지도 쓰지도 않습니다. resultMap 은 모르는 컬럼을 그냥 무시하고 INSERT 는 컬럼을 명시하므로 NULL 로 남습니다 — 이 절과 4절에 추가로 손댈 것이 없습니다.

5 컨트롤러 3개 + 화면 1개

카테고리 목록이 세 곳에 복제돼 있었고 값이 옛 4종이었습니다. 새 5종과 다르므로 그대로 두면 `ExceptionHandler` 가 기관 등록을 거부합니다.

KGI12000\$FileStatusList.java · **KGI12400\$ClassifiedFileList.java**

```
전 private static final List<String> ALL_CATEGORIES =  
    Arrays.asList("지자체", "대학교", "대학병원", "공공기관");  
...  
model.addAttribute("categories", ALL_CATEGORIES);
```

```
후 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

상수 선언과 `import java.util.Arrays;` 는 지웁니다.

KGI13000\$InstitutionList.java 화면에 목록을 넘기도록 추가

```
추가 import com.kb.uploader.code.InstitutionCategory;  
...  
model.addAttribute("categories", InstitutionCategory.names());
```

templates/institution.html

```
전 <option value="지자체">지자체</option>  
<option value="대학교">대학교</option>  
<option value="대학병원">대학병원</option>  
<option value="공공기관">공공기관</option>
```

```
후 <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
```

다른 화면(`upload.html` · `file-status.html` · `classified-rework.html`)은 이미 `${categories}` 를 쓰고 있어 손댈 것이 없습니다.

6 내부망에서 적용하는 법

방법 ㉓ — 파일 교체 권장

외부망 리포	내부망 WTP 폴더
uploader/src/main/java/com/kb/uploader/**/*.java	src/com/kb/uploader/**/*.java
uploader/src/main/resources/mapper/*.xml	src/mapper/*.xml
uploader/src/main/resources/schema-*.sql	src/schema-*.sql
uploader/src/main/resources/templates/institution.html	src/templates/institution.html

신규 폴더 `src/com/kb/uploader/code/` 가 생깁니다. 교체 후 **Project** → **Clean** → **Publish**.

방법 ㉔ — 매퍼를 손으로 고칠 때의 치환 순서

순서를 지키지 않으면 XML 이 깨집니다

INSTITUTION → TSKGIAF02 를 먼저 돌리면 INSTITUTION_NAME · INSTITUTION_SEQ 까지 망가집니다.
ID 는 databaseId · keyProperty 같은 속성 이름에도 들어 있어 무차별 치환하면 문서가 통째로 깨 집니다.

#	찾을 것	바꿀 것	주의
1	UPLOADED_FILE_SEQ	SQ_KGI_AGENTFILETABLE	가장 길어서 먼저
2	INSTITUTION_SEQ	SQ_KGI_AGENTGIGANTABLE	—
3	INSTITUTION_NAME	기관명	INSTITUTION 보다 먼저
4	ORIGINAL_NAME	원본파일명	—
5	CLASSIFIED_AT	분류일시	—
6	MODIFIED_AT	수정일시	—
7	UPLOADED_AT	업로드일시	—
8	STORED_PATH	저장경로	—
9	FILE_YEAR	문서년	—
10	UPLOADED_FILE	TSKGIAF01	1번을 먼저 했어야 안전
11	INSTITUTION	TSKGIAF02	2·3번을 먼저 했어야 안전
12	CATEGORY	기관영업분류코드	한 번만
13	STATUS	분류상태구분	한 번만
14	NAME	기관명	⚠ InstitutionMapper 에서만
15	ID	업로드파일일련번호 / 기관일련번호	🔴 일괄 치환 금지 — 한 곳씩 눈으로
16	그다음 TypeHandler 를 손으로 넣는다(3-1·3-2·4-1) — 치환으로는 안 됩니다		
17	그다음 리터럴을 코드로(3-5의 '04' · '03', 3-8의 '02')		

16·17번 때문에 ㉔는 권하지 않습니다. 파일 교체가 훨씬 안전합니다.

7 적용 후 확인

확인	방법	정상
컴파일	Project → Clean 후 Problems 뷰	오류 0건. <code>code</code> 패키지가 보임
배포	<code>WEB-INF/classes/mapper/*.xml</code> 열어보기	<code>TSKGIAF01</code> 이 보임
기동	서버 로그	TypeHandler 예외 없음
드롭다운	<code>/uploader/institutions</code>	분류 선택지가 5종(지방자치단체·공공기관·대학교·병원·법원)
코드 저장	테이블 생성 후 기관 1건 등록 → DB 조회	기관영업분류코드 에 <code>'01'</code> 같은 코드
일시 저장	파일 1건 업로드 → DB 조회	업로드일시 가 <code>20260908103000</code> 14자

별첨신규 자바 6개 — 소스 전문

파일을 그대로 복사하실 수 있으면 이 별첨은 보지 않으셔도 됩니다. 반입 경로가 막혀 손으로 옮겨 적어야 하거나, 반입 전 검수가 필요할 때 쓰십시오.

옮겨 적을 때 지킬 것 셋

- 파일 인코딩은 UTF-8 이어야 합니다. 주석·문자열에 한글이 많아 cp949 로 저장하면 통째로 깨집니다. (WTP 폴더는 `.settings/org.eclipse.core.resources.prefs` 에서 UTF-8 로 고정해 두었습니다.)
- 패키지 경로가 폴더와 맞아야 합니다. `package com.kb.uploader.code;` 인 파일은 `src/com/kb/uploader/code/` 에 있어야 합니다 — `code` 폴더는 새로 만들어야 합니다.
- 클래스 이름과 파일 이름이 같아야 합니다(대소문자 포함).

읽는 순서를 권한다면

`CodeTypeHandler` 하나만 읽으면 나머지 둘(`InstitutionCategory- · ClassificationStatusTypeHandler`)은 표만 같아 끼운 15줄짜리입니다.
`InstitutionCategory · ClassificationStatus` 는 코드표 그 자체이고,
`LocalDateTimeStringTypeHandler` 는 형식 변환만 합니다.

`src/com/kb/uploader/code/InstitutionCategory.java` 57줄 · 기관영업분류코드 코드표. 화면 드롭다운 목록의 유일한 출처이기도 하다.

```
package com.kb.uploader.code;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * 기관영업분류코드 - 사내 인포타임 {@code 코드2} 로 저장하는 코드값.
 *
 * <p><b>왜 코드인가.</b> 사내 DB 표준의 {@code 구분코드} 는 <b>5자리 이하</b>여야 한다
 * (DBA 정책). 그런데 우리가 쓰던 값 {@code 지방자치단체} 는 6자리 들어가지 않는다.
 * 그래서 DB 에는 {@code '01'} 을 넣고, <b>자바와 화면은 한글 이름을 그대로 쓴다.</b>
 * 변환은 {@code InstitutionCategoryTypeHandler} 한 곳에서만 일어난다 -
 * 도메인·서비스·컨트롤러·Thymeleaf 는 이 코드의 존재를 모른다.
 *
 * <p>⚠ <b>여기가 카테고리 목록의 유일한 출처다.</b> 종전에는 같은 목록이
 * {@code KGI12000}·{@code KGI12400} 두 컨트롤러와 {@code institution.html} 에
 * <b>세 벌</b>로 복제돼 있었고, 그래서 옛 4종(지자체·대학교·대학병원·공공기관)이
 * 그대로 남아 있었다. 늘리거나 바꿀 때는 이 파일만 고친다.
 *
 * <p>⚠ 코드값을 바꾸면 <b>이미 저장된 행을 함께 옮겨야 한다.</b> 지금은 테이블이
 * 비어 있어 자유롭지만, 운영 뒤에는 UPDATE 가 따른다.
 */
public final class InstitutionCategory {

    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();

    static {
        put("01", "지방자치단체");
        put("02", "공공기관");
        put("03", "대학교");
        put("04", "병원");
    }
}
```

```

        put("05", "법원");
    }

    private static void put(String code, String name) {
        NAME_TO_CODE.put(name, code);
        CODE_TO_NAME.put(code, name);
    }

    /** 화면 드롭다운에 그대로 쓰는 순서다(코드 오름차순). */
    public static List<String> names() {
        return Collections.unmodifiableList(new java.util.ArrayList<String>(NAME_TO_CODE.keySet()));
    }

    public static Map<String, String> nameToCode() {
        return Collections.unmodifiableMap(NAME_TO_CODE);
    }

    public static Map<String, String> codeToName() {
        return Collections.unmodifiableMap(CODE_TO_NAME);
    }

    private InstitutionCategory() {}
}

```

src/com/kb/uploader/code/ClassificationStatus.java 51줄 · 분류상태 코드표 + 자바가 쓰는 상태 상수.

```

package com.kb.uploader.code;

import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * 분류상태 - 사내 인포타임 {@code 구분코드2} 로 저장하는 코드값.
 *
 * <p>자바가 쓰는 값은 {@code "UNCLASSIFIED"} 처럼 <b>종전 그대로</b>다
 * ({@code UploadedFile.classify()}·{@code softDelete()} 등). 12자리 5자리 상한을
 * 넘으므로 DB 에만 코드로 바꿔 넣는다 - 변환은
 * {@code ClassificationStatusTypeHandler} 한 곳뿐이다.
 *
 * <p>⚠ <b>Mapper XML 의 SQL 리터럴은 이 변환을 타지 않는다.</b>
 * TypeHandler 는 <b>파라미터와 결과</b>에만 걸리므로,
 * {@code SET 분류상태구분 = '04'} 처럼 문장에 직접 박힌 값은 <b>코드로 적어야 한다.</b>
 * 그 자리에 {@code 'DELETED'} 를 적으면 조용히 아무 행동 안 바뀐다.
 */
public final class ClassificationStatus {

    public static final String UNCLASSIFIED = "UNCLASSIFIED";
    public static final String CLASSIFIED   = "CLASSIFIED";
    public static final String REJECTED     = "REJECTED";
    public static final String DELETED      = "DELETED";

    private static final Map<String, String> NAME_TO_CODE = new LinkedHashMap<String, String>();
    private static final Map<String, String> CODE_TO_NAME = new LinkedHashMap<String, String>();

    static {
        put("01", UNCLASSIFIED);
        put("02", CLASSIFIED);
        put("03", REJECTED);
        put("04", DELETED);
    }

    private static void put(String code, String name) {
        NAME_TO_CODE.put(name, code);
        CODE_TO_NAME.put(code, name);
    }

    public static Map<String, String> nameToCode() {
        return Collections.unmodifiableMap(NAME_TO_CODE);
    }
}

```

```

    public static Map<String, String> codeToName() {
        return Collections.unmodifiableMap(CODE_TO_NAME);
    }

    private ClassificationStatus() {}
}

```

src/com/kb/uploader/config/CodeTypeHandler.java 74줄 · 코드 변환 공통 뼈대(추상). 실질적인 로직은 여기 하나뿐이고 아래 둘은 표만 갈아 끼운다.

```

package com.kb.uploader.config;

import org.apache.ibatis.type.BaseTypeHandler;
import org.apache.ibatis.type.JdbcType;

import java.sql.CallableStatement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Map;

/**
 * 자바가 쓰는 <b>이름</b>과 DB 에 들어가는 <b>코드</b>를 잇는다.
 *
 * <p><b>왜 두는가.</b> 사내 DB 표준의 {@code 구분코드} 는 5자리 이하여야 하는데
 * 우리가 쓰는 값({@code UNCLASSIFIED}·{@code 지방자치단체})은 그보다 길다.
 * 값을 코드로 바꾸면 도메인·서비스·컨트롤러·화면·테스트가 전부 바뀌므로,
 * <b>변환을 여기 한 곳에 가둔다.</b> 그 덕에 자바 쪽은 종전 값을 그대로 쓴다.
 *
 * <p>⚠ <b>모르는 값이 오면 조용히 넘기지 않고 예외를 던진다.</b> 코드가 안 맞으면
 * 증상이 "저장은 됐는데 조회가 안 됨" 처럼 엉뚱하게 나타나기 때문이다.
 *
 * <p>⚠ 두 하위 구현 모두 자바 타입이 {@code String} 이라 <b>자동 등록(type-handlers-package)
 * 을 쓸 수 없다</b> - 같은 타입에 둘이 걸려 어느 쪽이 잡힐지 알 수 없다.
 * 그래서 Mapper XML 에서 {@code typeHandler=} 로 <b>자리마다 명시</b>한다.
 */
public abstract class CodeTypeHandler<String> {

    protected abstract Map<String, String> nameToCode();

    protected abstract Map<String, String> codeToName();

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, String name, JdbcType jdbcType)
        throws SQLException {
        String code = nameToCode().get(name);
        if (code == null) {
            throw new SQLException(getClass().getSimpleName()
                + ": 코드로 옮길 수 없는 값이다 - '" + name + "'. 아는 값: " + nameToCode().keySet());
        }
        ps.setString(i, code);
    }

    @Override
    public String getNullableResult(ResultSet rs, String columnName) throws SQLException {
        return toName(rs.getString(columnName));
    }

    @Override
    public String getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
        return toName(rs.getString(columnIndex));
    }

    @Override
    public String getNullableResult(CallableStatement cs, int columnIndex) throws SQLException {
        return toName(cs.getString(columnIndex));
    }
}

```

```

    private String toName(String code) throws SQLException {
        if (code == null) {
            return null;
        }
        String trimmed = code.trim();
        if (trimmed.isEmpty()) {
            return null;
        }
        String name = codeToName().get(trimmed);
        if (name == null) {
            throw new SQLException(getClass().getSimpleName()
                + ": DB 에 모르는 코드가 들어 있다 - '" + trimmed + "'. 아는 코드: " + codeToName().keySet());
        }
        return name;
    }
}

```

src/com/kb/uploader/config/InstitutionCategoryTypeHandler.java 19줄 · 위 뼈대 + 기관영업분류 표.

```

package com.kb.uploader.config;

import com.kb.uploader.code.InstitutionCategory;

import java.util.Map;

/** {@code 지방자치단체} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
public class InstitutionCategoryTypeHandler extends CodeTypeHandler {

    @Override
    protected Map<String, String> nameToCode() {
        return InstitutionCategory.nameToCode();
    }

    @Override
    protected Map<String, String> codeToName() {
        return InstitutionCategory.codeToName();
    }
}

```

src/com/kb/uploader/config/ClassificationStatusTypeHandler.java 19줄 · 위 뼈대 + 분류상태 표.

```

package com.kb.uploader.config;

import com.kb.uploader.code.ClassificationStatus;

import java.util.Map;

/** {@code UNCLASSIFIED} ↔ {@code '01'} - 자세한 이유는 {@link CodeTypeHandler}. */
public class ClassificationStatusTypeHandler extends CodeTypeHandler {

    @Override
    protected Map<String, String> nameToCode() {
        return ClassificationStatus.nameToCode();
    }

    @Override
    protected Map<String, String> codeToName() {
        return ClassificationStatus.codeToName();
    }
}

```

src/com/kb/uploader/config/LocalDateTimeStringTypeHandler.java 70줄 · LocalDateTime ↔

YYYYMMDDHH24MISS 문자열.

```

package com.kb.uploader.config;

```

```

import org.apache.ibatis.type.BaseTypeHandler;
import org.apache.ibatis.type.JdbcType;

import java.sql.CallableStatement;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

/**
 * 자바 {@link LocalDateTime} 과 사내 인포타임 {@code 일시20}(CHARACTER 20) 을 잇는다.
 *
 * <p>저장 형식은 <b>{@code YYYYMMDDHH24MISS}</b> 14자다(사내 표준). 컬럼이 20자인 것은
 * 여유일 뿐이라 우리는 14자만 채운다.
 *
 * <p><b>왜 두는가.</b> 표준이 시각을 문자열로 두라고 하는데 도메인은
 * {@code LocalDateTime} 이다. 도메인을 문자열로 바꾸면 {@code UploadedFile}·서비스·
 * 화면·테스트가 전부 바뀌므로, <b>변환을 여기 한 곳에 가둔다.</b>
 *
 * <p>⚠ 읽을 때 {@code trim()} 한다. 컬럼이 고정폭 {@code CHAR} 로 만들어지면
 * 뒤에 공백이 붙어 오는데, 그대로 파싱하면 {@code DateTimeParseException} 이 난다.
 *
 * <p>⚠ <b>정렬이 문자열 정렬</b>이 된다. {@code YYYYMMDDHH24MISS} 는 자릿수가 고정이라
 * 사전순 = 시간순이 맞지만, 형식을 바꾸면 그 성질이 깨진다
 * ({@code ORDER BY 업로드일시 DESC} 가 조용히 어긋난다).
 */
public class LocalDateTimeStringTypeHandler extends BaseTypeHandler<LocalDateTime> {

    private static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("yyyyMMddHHmmss");

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i, LocalDateTime parameter, JdbcType jdbcType)
        throws SQLException {
        ps.setString(i, parameter.format(FORMAT));
    }

    @Override
    public LocalDateTime getNullableResult(ResultSet rs, String columnName) throws SQLException {
        return parse(rs.getString(columnName));
    }

    @Override
    public LocalDateTime getNullableResult(ResultSet rs, int columnIndex) throws SQLException {
        return parse(rs.getString(columnIndex));
    }

    @Override
    public LocalDateTime getNullableResult(CallableStatement cs, int columnIndex) throws SQLException {
        return parse(cs.getString(columnIndex));
    }

    private LocalDateTime parse(String raw) throws SQLException {
        if (raw == null) {
            return null;
        }
        String value = raw.trim();
        if (value.isEmpty()) {
            return null;
        }
        try {
            return LocalDateTime.parse(value, FORMAT);
        } catch (DateTimeParseException e) {
            throw new SQLException("일시 형식이 YYYYMMDDHH24MISS 가 아니다 - '" + value + "'", e);
        }
    }
}

```

② uploader 소스 변경 상세 · 2026-09-08 확정본 · 선행 문서: [내부망작업지시_2026-09-08_1_스키마변경.pdf](#)